Interactive Analytics Dashboard for Google Play Store Applications

Internship Project Report

Submitted by: Shivraj Dash

Programme: Bachelor of Business Administration (BBA) – Business Analytics, Semester IV

[Manipal University Jaipur]

Under the guidance of: [SATYA PRAKASH MOHANTY (MENTOR) ELEVANCE SKILLS]

Date of Submission: [14 JULY 2026]

Table of Contents

(Right-click below and choose "Update Field" — or press Ctrl+A then F9 — to populate page numbers.)

1. Dataset Description

The analysis is built on the Google Play Store Apps dataset, a publicly available dataset widely used for mobile application analytics. The project draws on two related files: a main listing of individual Play Store applications, and a second file of user reviews for a subset of those applications, including sentiment scores generated from the review text. In its most commonly distributed form, the main file holds a little over ten thousand app records across thirteen columns, and the reviews file holds tens of thousands of individual review rows across five columns — though the exact counts in any one notebook depend on which rows survive that notebook's own cleaning and filtering steps. One inconsistency worth flagging up front: five of the six notebooks load the main file as "Play Store Data.csv" (with a space), while the sixth (Task 5) loads it as "Play_Store_Data.csv" (with an underscore); the same split applies to the reviews file. Running all six notebooks from one shared data folder means keeping both filename variants of each file, or renaming them, since only Task 5 uses the underscored names.

The fields below recur across the six tasks. Not all of them are used by every notebook — Genres and Current Version, for instance, are present in the raw file but are not filtered on by any of the six tasks — but all are part of describing what the dataset contains.

Category: the classification Google assigns to each app on the Play Store, such as GAME, BUSINESS, or COMMUNICATION. This field drives the grouping and filtering logic in every one of the six tasks.

App: the display name of the application. Several tasks filter on the text of this field directly — for example, excluding names that start with a particular letter or contain a particular character.

Rating: the app's average user rating on a scale of 1 to 5. Four of the six tasks (1, 5, and 6, plus a plain sanity check in Task 5) require a minimum rating, and three also discard any rating recorded above 5, which would be a data error rather than a real score.

Reviews: the total number of user reviews the app has received. This is used both as a reliability filter — screening out apps with too little feedback to draw a conclusion from — and, in Task 1, as a chart metric in its own right.

Size: the app's storage size, stored in the raw file as text such as "19M" or "509k". Three tasks (1, 5, 6) convert this into a single numeric megabyte scale before it can be filtered or plotted; a fourth (Task 3) takes a simpler route and keeps only the rows already recorded in megabytes.

Installs: the number of times the app has been installed, stored as text with commas and a trailing plus sign (for example "10,000+"). This is the core popularity metric behind every one of the six charts.

Price: the app's listed price, stored as text with a leading dollar sign for paid apps. Task 3 is the only one of the six that uses this field, converting it to a number and multiplying it by Installs to derive a Revenue figure.

Content Rating: the age-appropriateness classification of the app, such as "Everyone" or "Teen." Task 3 restricts its comparison to apps rated "Everyone."

Genres: a finer-grained classification than Category, sometimes combining more than one genre for a single app. Present in the raw file but not used by any of the six tasks.

Last Updated: the date the app's Play Store listing was last revised. Because the dataset has no true install-history field, three tasks (1, 4, 6) use this date as a stand-in time axis — either to filter on a specific month (Task 1) or to build a monthly trend line (Tasks 4 and 6).

Current Version / Android Version: the app's own version number and the minimum Android OS version it supports, both stored as free-text strings. Current Version is not used by any of the six tasks; Android Version is parsed in Task 3, which pulls out the first numeric version it can find (for example, reading "4.1" out of "4.1 and up") and compares it against a threshold.

Sentiment_Subjectivity: taken from the reviews file, this score (0 to 1) measures how opinion-based versus fact-based a review's language is. Task 5 is the only one of the six that uses it, averaging it per app and requiring apps whose reviews lean opinion-based.

Before any chart is built, every notebook runs a broadly similar sequence: rows missing values in the columns that task depends on are dropped, numeric-looking text fields are converted with pandas' to_numeric and any value that fails to parse is coerced to missing rather than left as text, Installs is stripped of commas and plus signs before conversion, Size is normalised to megabytes, and — where the task needs it — Android Version is reduced to its first numeric component and Last Updated is parsed into an actual date. Two of the six notebooks (Tasks 4 and 6) also remove exact duplicate rows before anything else runs, which the other four do not do explicitly.

2. Project Overview

The goal of this project is to build six interactive analytics views over the Google Play Store dataset, each answering a different business question about app performance, and to present them as a single dashboard experience. Rather than one static chart, each visualization is its own self-contained module: its own data pipeline, its own filters, its own chart, and its own visibility rule tied to the time of day in India Standard Time (IST). A visitor who opens a given dashboard page at a particular moment only sees that page's chart if the current IST time falls inside its assigned window; outside that window, the chart area is replaced with a short notice instead.

This time-based visibility is not a decoration added on top of the analysis — it is the organising idea of the project. It forces every one of the six tasks to be built as an independent, self-sufficient unit, complete with its own fallback states for when there is no data left after filtering and for when the viewing window has closed. Combined with the category-label translations used in three of the six tasks, and the country-level modelling built specifically for Task 2's map, the result behaves less like a fixed report and more like a small interactive product — one whose content changes depending on when, and in one case how, it is being viewed.

The purpose of building this kind of interactive analytics, rather than a fixed set of charts, is to practise the full loop of a real analytics deliverable: cleaning a messy public dataset, deciding what specific business question each chart should answer, applying the filtering logic needed to answer it cleanly, and packaging the result in a form a non-technical stakeholder could open in a browser and understand without reading any code.

3. Work Summary

The project started from a shared understanding of the Google Play Store dataset: what each column meant, which fields were reliable enough to filter on, and which needed cleaning before they could be used at all. From there, the work followed a similar rhythm across the six tasks. Data preprocessing came first — loading the relevant CSV file or files, dropping rows with missing values in the columns that task needed, and converting text fields such as Installs, Size, Price, and Android Version into proper numeric types. Data cleaning and transformation went hand in hand with this, since almost none of the raw columns could be filtered or plotted directly: Installs needed its commas and plus signs stripped, Size needed its "M" and "k" suffixes converted onto one megabyte scale, and Android Version needed its numeric component pulled out of a longer text string with a regular expression.

Once a task's data was clean, that task's specific filtering logic was applied — rating, install, review, or revenue thresholds, category restrictions, or conditions on the app name's text — and, where the task called for it, the top categories or apps were identified by sorting on the relevant metric. Dashboard development followed a broadly similar pattern across the six tasks: build the chart in Plotly or Matplotlib, wrap it in an HTML page (dark-themed in five of the six cases), and gate its visibility behind an IST time check computed with the pytz library so the chart only appears during its assigned window. Three of the six tasks additionally translate specific category labels into another language for the legend or hover text — Hindi, Tamil, and German in Task 4; Hindi and Tamil (with the German entry for Dating left unused) in Task 5; and French, Spanish, and Japanese in Task 6 — using a small mapping dictionary that substitutes a translated label wherever that category's name would otherwise be shown, without touching how the data is filtered or grouped. Two tasks (4 and 6) go a step further and highlight periods of unusually fast month-over-month growth directly on the chart, by redrawing that segment bolder and more opaque than the rest of the line or band.

One methodological note applies to the six task chapters that follow. None of the six notebooks were executed against the dataset while this report was being prepared, and none of them print or save the specific values their filters would produce — exact top categories, row counts, or install totals. The Insights subsection in each task chapter below therefore describes what that chart is built to reveal and how its own filters shape that view, rather than quoting figures the notebooks have not actually generated. Sharing the executed notebooks, their printed output, or the underlying CSV files would let this section be revisited with concrete numbers in place of the current design-level description.

4. Task 1 — Grouped Bar Chart: Top Categories by Rating and Reviews

4.1 Task Objective

Task 1 looks at whether the categories with the largest install bases are also the categories users rate highly, or whether the two measures move independently. It does this by taking the ten categories with the highest total installs, after filtering, and comparing each one's average rating against its total review volume side by side. In business terms, this is a quick way of asking whether popularity, engagement, and quality actually line up with each other, which matters for judging whether a category is a genuinely strong target for new app development or simply a crowded one.

4.2 Data Filtering

Before any ranking happens, the notebook drops rows missing Category, Rating, Reviews, Installs, Size, or Last Updated, and separately screens out rows where one of those fields is only a blank string once stripped of whitespace — a check a plain missing-value test would not catch. From the rows that remain, three conditions apply: Rating of 4.0 or above, Size greater than 10 MB, and a Last Updated date falling in the month of January, in any year present in the data, since only the month component is checked. The ten categories shown in the chart are the ten with the highest total Installs among the rows left after all three filters.

4.3 Visualization Used

The chart is a dark-themed, grouped Plotly bar chart. Each of the ten categories gets two bars side by side: one for average rating, coloured blue, and one for total review count, coloured red. Because review totals can run into the hundreds of thousands while rating sits on a 0-to-5 scale, the review-count bar is expressed in millions rather than plotted on a second axis, which keeps both bars readable on one shared scale. Each bar carries its exact value as a text label above it, and category names sit along the x-axis at a slant so all ten fit without overlapping. The notebook loads a second file of user reviews at the very start but never actually uses it — the entire chart is built from the main app-listing file alone.

4.4 Dashboard Time Restriction

The chart is visible only between 3:00 PM and 5:00 PM IST, inclusive of both endpoints. As in every task, the current IST time is computed with the pytz library rather than relying on the machine's local timezone, so the check gives the same answer regardless of where the notebook happens to run.

4.5 Implementation Explanation

After the shared type conversions, Last Updated is parsed into an actual date with pd.to_datetime and checked against month 1. The filtered rows are then grouped by Category once, computing three aggregates together — mean Rating, summed Reviews, and summed Installs — and that same table

is sorted by summed Installs to take the top ten rows. Because that one grouped table already holds both the rating and review figures the chart needs, no second grouping step is required. The review totals are divided by one million and rounded to two decimal places only at the point the chart is drawn, so the underlying data still carries the exact review count throughout.

4.6 Insights

Because the filters require a January update and a size above 10 MB in the same pass, the ten categories being compared are a fairly specific slice of the catalogue — apps that were both actively maintained in a particular month and not minimal in size — rather than every app in each category, so a category's position in this top ten is best read as characteristic of that slice rather than of the category as a whole. Putting rating and review volume side by side is built to answer a specific question: whether the categories that install the most also hold up on quality and engagement, or whether high installs can coexist with comparatively thin review activity or a middling rating.

5. Task 2 — Choropleth Map: Global Installs by Category

5.1 Task Objective

Task 2 visualizes how installs for a set of major categories are spread across a set of world markets, using a map rather than a bar chart or table. The intent is to give a more immediate sense of category reach than a ranked list would, and to flag which category-market combinations cross a high-install threshold.

5.2 Data Filtering

Category and Installs are the two columns this task strictly needs, so those are the two checked for missing or blank values before anything else runs. Categories whose name starts with A, C, G, or S are removed first — which rules out GAME, COMMUNICATION, SOCIAL, and SPORTS, among others, regardless of their install totals — and only afterward are the five remaining categories with the highest total installs selected. A separate 1,000,000-install threshold is applied later, not to remove rows, but to flag which country-category combinations get labelled "High Install Category" versus "Normal" in the map and its summary table.

5.3 Visualization Used

The chart is an interactive Plotly choropleth map, and it is the only one of the six visualizations built with a dropdown selector: one map trace is created per top-five category, only the first is visible when the page loads, and a dropdown above the map switches which trace is shown. The Play Store dataset does not record installs by country for any app — only a single global figure — so the notebook builds a country split itself: a fixed dictionary assigns twenty-five countries a rough share of installs (India and the United States get the largest shares, most others one to four percent), a small amount of random noise is added per category so the shares are not identical across categories, and each category's total installs are then divided across those twenty-five countries according to that weighting. This is a deliberate way of making a geographic visualization possible despite the source data containing no geography, and it should be read as exactly that — an illustrative distribution built for this chart, not a claim about where installs actually originate. Each category's map uses its own colour scale, and hovering over a country shows its name, category, exact install figure, and high/normal status. Unlike the other five dashboards in this project, this one uses a light background rather than a dark theme.

5.4 Dashboard Time Restriction

The chart is visible between 6:00 PM and 8:00 PM IST (up to, but not including, 8:00 PM exactly), computed with pytz as in the other tasks. This dashboard goes one step further than the other five: alongside the Python-computed check baked into the saved HTML at build time, the page also carries its own small JavaScript clock that recalculates the IST time in the visitor's browser every ten seconds and toggles the map and the time notice accordingly. That means this particular page can flip from

"not yet visible" to "visible" on its own while someone has it open, where the other five dashboards only reflect whatever the time happened to be when their notebook was last run.

5.5 Implementation Explanation

Once Installs is cleaned into a numeric column, categories are grouped and summed to rank them, the excluded first letters are removed, and the top five surviving categories are kept. For each of those five, the notebook distributes its total installs across the fixed country list using a seeded random-number generator, so the same "random" split reproduces on every run, and records whether each resulting country figure clears the one-million mark. The choropleth is built with one trace per category rather than one combined trace, which is what makes the dropdown-driven view possible: only the active trace's visibility is set to true, and switching categories in the dropdown just flips which trace is shown rather than redrawing the map. One figure on the dashboard — the "#1 Category" stat card — is written into the page as the fixed text "Productivity" rather than being calculated from the category totals the way the other stat cards on the same page are, so it would not automatically update if the underlying ranking changed on a future run with different data.

5.6 Insights

Excluding categories starting with A, C, G, or S removes several of the Play Store's largest categories by definition, GAME and SOCIAL among them, so this map is not built to find the single largest category overall — it surfaces categories that are large within the remaining alphabet, which can highlight categories a simple top-five-overall ranking would otherwise crowd out. Because the country-level split is a modelling choice rather than data the dataset actually contains, the map is best read as a way of visualising the relative scale of these five categories through a geographic lens, not as a literal claim about which countries drive each category's installs.

6. Task 3 — Dual-Axis Chart: Free vs Paid Installs and Revenue

6.1 Task Objective

Task 3 asks a monetisation question: within the categories that generate the most installs, do Paid and Free apps differ in the revenue and installs they typically produce? The chart isolates the top three categories by installs and compares Free versus Paid apps within each one, using both installs and revenue as the basis for comparison.

6.2 Data Filtering

Before revenue can be calculated, Price is cleaned by stripping the dollar sign and treating anything unparsable as zero, and Revenue is computed for every app as Price multiplied by Installs. Rows must then meet all of the following: Installs of at least 10,000, Revenue of at least $10,000, a minimum supported Android version above 4.0, an app size above 15 MB (using only rows whose Size is already recorded in megabytes, so kilobyte-sized apps are excluded before this threshold is even applied), and a Content Rating of exactly "Everyone." App name length is capped at 30 characters, counting every character including spaces and symbols. Finally, only rows where Type is cleanly "Free" or "Paid" are kept, guarding against any stray value in that column reaching the Free/Paid comparison. The top three categories used in the chart are the three with the highest total Installs among the rows that survive every one of these filters.

6.3 Visualization Used

The chart is a dual-axis Plotly figure combining a bar trace and a line trace. For each of the three categories, average Installs for Free apps and for Paid apps are plotted as bars on the left-hand axis, while average Revenue for the same six category-and-type combinations is plotted as a line with markers on a separate axis on the right, since revenue and install counts sit on completely different scales. Free apps are coloured green and Paid apps orange in the bars, and the revenue line is a contrasting purple. Hovering over any bar or point shows the exact category, type, and value. The page around the chart also includes a small table listing the top three categories and their total installs, which stays visible regardless of the time window, so a visitor always sees at least the category ranking even outside the chart's active hours.

6.4 Dashboard Time Restriction

The chart is visible only from 1:00 PM up to, but not including, 2:00 PM IST. Outside that window, the chart container is not rendered into the page at all, and a short notice states the visible hours instead.

6.5 Implementation Explanation

The notebook first drops rows missing any of Category, Installs, Size, Android Ver, Type, or Content Rating, then cleans Installs and Price into numeric form and derives Revenue as their product. Size is

restricted to values ending in "M" before conversion, and Android Ver is reduced to its leading numeric version with a regular expression, since the raw field can contain ranges or the text "Varies with device." App name length is computed as the plain length of the App string. All six filter conditions are applied together as one combined boolean mask. The top three categories are found by grouping the filtered data by Category and summing Installs. The chart data itself comes from a second grouping, this time by both Category and Type together, computing the mean of Installs and the mean of Revenue for each of the six resulting groups, which is what the bar-and-line chart plots.

6.6 Insights

The chart is built specifically to test whether Paid apps, despite typically reaching far fewer installs than Free apps, make up for it in average revenue per app — a standard trade-off in mobile monetisation between reach and direct payment. Because every app in this comparison has already cleared a $10,000 revenue minimum and a 10,000-install minimum, the chart is not comparing typical apps against each other; it is comparing successful Free apps against successful Paid apps within categories that are already installs leaders, which is a more demanding and more informative comparison than an unfiltered Free-versus-Paid split would be.

7. Task 4 — Time Series Line Chart: Installs Trend and Growth Spikes

7.1 Task Objective

Task 4 tracks how installs trend over time for a specific slice of categories, and calls out the months where that growth was unusually fast. Since the raw dataset has no real install-history field, the chart uses each app's Last Updated date as a stand-in timeline, letting the notebook approximate a trend the source data does not directly provide.

7.2 Data Filtering

After removing duplicate rows and dropping any row missing App, Category, Reviews, Installs, or Last Updated, the notebook keeps only apps with more than 500 Reviews, excludes apps whose name starts with X, Y, or Z, and excludes apps whose name contains the letter S anywhere — both name checks are case-insensitive. On category, the filter keeps any category starting with E, C, or B, and separately keeps Dating even though "Dating" does not start with any of those three letters; the condition is written as an explicit or-clause rather than a strict starts-with rule, which is why Dating appears in this chart's legend alongside Business, Beauty, Comics, Communication, Education, Entertainment, and Events.

7.3 Visualization Used

The chart is a Matplotlib line chart, one coloured line per category, each plotted against a monthly timeline built from Last Updated. Wherever a category's month-over-month growth exceeds 20%, that segment of its line is redrawn thicker and at full opacity, with a light fill added beneath it, so a fast-growth month stands out without needing to be read off the axis. Three of the nine categories get translated legend labels — Beauty in Hindi, Business in Tamil, and Dating in German — and the notebook explicitly sets a font-family fallback chain (Noto Sans, then Noto Sans Devanagari, then Noto Sans Tamil) before drawing the chart, because Matplotlib needs to be told which fonts can render non-Latin scripts or the Hindi and Tamil text would appear as missing-character boxes. Once drawn, the chart is saved as a PNG in memory and embedded into the dashboard page as a base64-encoded image rather than as an interactive figure.

7.4 Dashboard Time Restriction

The chart is visible only between 6:00 PM and 9:00 PM IST, the longest of the six visibility windows in the project, checked the same way as the other tasks using pytz.

7.5 Implementation Explanation

Reviews and Installs go through the same cleaning pattern used throughout the project, and Last Updated is converted to a date and reduced to a year-month period. The filtered rows are grouped by that period and by Category, summed on Installs, and pivoted so each category becomes its own

column across a shared monthly timeline, with any month a category has no data for filled to zero. Month-over-month growth is the percentage change between consecutive rows of that pivoted table for each category, with any division-by-zero result replaced by a missing value so it is not mistaken for a real spike. When the chart is drawn, every growth value above 20% triggers a second, bolder pass over that specific line segment, drawn on top of the normal line for that category.

7.6 Insights

Because Last Updated stands in for a true installs-over-time history, the chart should be read as showing how installs among currently tracked apps break down across the months those apps were last revised, not as a literal day-by-day install count the way a live analytics feed would show. Including Dating alongside the E/C/B-starting categories through an explicit exception, rather than tightening the category rule to genuinely match only E, C, and B, is a deliberate choice that broadens the comparison to nine categories spanning utility apps (Communication, Education, Business), leisure apps (Comics, Entertainment, Events), and personal-interest apps (Beauty, Dating) — a wider spread than a strict starts-with-E/C/B filter alone would have produced.

8. Task 5 — Bubble Chart: App Size vs Average Rating

8.1 Task Objective

Task 5 looks at the relationship between how large an app is and how highly it is rated, bringing installs into the same chart as bubble size. The goal is to see whether bigger apps tend to be rated better or worse than smaller ones, across a defined set of categories, without losing sight of which of those apps are actually popular.

8.2 Data Filtering

The task merges the main app listing with review-level sentiment data: subjectivity scores are averaged per app in the reviews file and joined onto the main data, keeping only apps that have at least one matching review, since the join is an inner join. From there, rows are kept only if Rating is above 3.5, Reviews exceed 500, Installs exceed 50,000, average Sentiment Subjectivity exceeds 0.5, the app name does not contain the letter "s" in either case, and the app's Category is one of nine allowed values: Game, Beauty, Business, Comics, Communication, Dating, Entertainment, Social, and Events.

8.3 Visualization Used

The chart is an interactive Plotly bubble chart, with one marker per app: app size on the x-axis, average rating on the y-axis, and bubble size scaled to the square root of installs, which keeps a handful of extremely high-install apps from producing bubbles large enough to swamp the rest of the chart. Every app in the Game category is coloured pink to make it stand out; the other eight categories are each assigned a distinct colour from a standard palette. Hovering over any bubble shows the app's name, its category label, rating, size, install count, and review count.

8.4 Dashboard Time Restriction

The chart is visible only between 5:00 PM and 7:00 PM IST, inclusive of both endpoints, based on the current IST time computed with pytz.

8.5 Implementation Explanation

After the standard cleaning of Rating, Reviews, Installs, and Size, the reviews file is cleaned separately and averaged per app before being merged onto the main data. All six filter conditions are then applied as a sequence of boolean masks. A Category_Label column is added by mapping Beauty to its Hindi label and Business to its Tamil label, with every other category falling back to its original English name. The map does include an entry intended to render Dating in German, but as written it maps Dating to the literal string "Dating," so Dating displays in English in the current version of this chart — unlike Task 4, where the equivalent Beauty/Business/Dating translation dictionary is complete and Dating does render in German. For the chart itself, marker size is computed as the square root of Installs

divided by a fixed constant, and each category is drawn as its own Plotly trace so the legend, colour, and hover template can all be controlled per category.

8.6 Insights

Combining a strict install minimum with a subjectivity-score minimum means this chart is deliberately restricted to apps that are both popular and have generated opinion-heavy reviews, rather than apps with a handful of terse or purely factual reviews, so the size-versus-rating relationship it shows should be read as characteristic of apps people have engaged with and formed opinions about, not the full population of apps in these nine categories. Pulling Game out in pink only makes sense if Game apps are expected to behave differently from the other eight categories on this axis, which is a reasonable expectation given how differently games are typically sized and rated compared to utility-oriented categories like Business or Communication — the colour choice invites a direct visual comparison between the Game cluster and everything else rather than requiring a reader to check the legend for every point.

9. Task 6 — Stacked Area Chart: Cumulative Installs Over Time

9.1 Task Objective

Task 6 extends the same time-trend idea from Task 4 into a cumulative view: instead of a separate line per category, it stacks each category's installs on top of the others so a viewer can see both how each category is growing and how the categories compare to each other in total, in one chart. It focuses on a set of categories where slow, steady adoption is more typical than the sharper spikes seen in more viral app categories.

9.2 Data Filtering

After de-duplicating rows and dropping any missing App, Category, Rating, Reviews, Size, or Last Updated, the notebook keeps apps rated 4.2 or above, excludes any app whose name contains a digit, keeps only categories starting with T or P, requires more than 1,000 Reviews, and keeps Size between 20 and 80 MB inclusive. The starts-with-T-or-P rule surfaces six categories in the source data — Tools, Travel & Local, Personalization, Productivity, Parenting, and Photography — three of which get translated legend labels.

9.3 Visualization Used

The chart is a Matplotlib stacked area chart built with stackplot, using cumulative installs for each category — a running total across months, not the installs added in any single month — so the top edge of the stack reflects combined installs to date across all six categories. Travel & Local is labelled in French, Productivity in Spanish, and Photography in Japanese; a font family capable of rendering the Japanese legend text is set explicitly before the chart is drawn, for the same reason Task 4 needs a font fallback chain for Hindi and Tamil. Wherever a category's month-over-month growth, measured on its non-cumulative monthly installs rather than the cumulative curve, exceeds 25%, that month's slice of its band is redrawn at full opacity against the surrounding band's lighter default, producing the increasing-colour-intensity effect for fast-growth months.

9.4 Dashboard Time Restriction

The chart is visible only between 4:00 PM and 6:00 PM IST, inclusive of both endpoints, checked the same way as the rest of the project.

9.5 Implementation Explanation

Size is normalised the same way as in Tasks 1 and 5, and Last Updated becomes a year-month period exactly as in Task 4. Installs are grouped and summed by category and month, pivoted into one column per category, and then run through a cumulative sum down each column to turn monthly totals into a running total. Growth is computed separately, as the month-over-month percentage change in the original, non-cumulative monthly totals, so a category that is still growing steadily but not accelerating

is not flagged just because its cumulative total is naturally always rising. To draw the highlighted bands, the notebook also computes the cumulative total across categories at each month, which gives it the top and bottom edge of every category's slice at every point in time — those edges are what get redrawn at full opacity for any month a category's growth exceeds 25%.

9.6 Insights

Because every band in the stack is cumulative, it only ever grows, so the useful comparison is the steepness of each band's slope and where its full-opacity, high-growth segments fall, rather than whether any band shrinks. Travel & Local, Productivity, and Photography, plus Tools, Personalization, and Parenting where they also clear the T/P and other filters, are categories generally associated with steady, repeat-use adoption rather than short bursts of virality, so a comparatively smooth stack shape here would be consistent with what these categories are typically expected to look like.

10. Dashboard Section

The six visualizations do not share a single dashboard shell — each notebook produces and saves its own HTML file — but five of the six follow a near-identical layout: a dark-themed page header naming the chart, a subtitle summarising the active filters in plain language, and a chart area that is either the rendered chart, a "no data" message, or a waiting notice, depending on conditions at the moment the notebook was run. Task 2's choropleth dashboard is the exception on two counts: it uses a light background rather than the dark theme the other five share, and it is the only one of the six that re-checks the time window live in the visitor's browser, through a small JavaScript clock, rather than relying solely on the state baked in at build time — so it can flip from "not yet visible" to "visible" on its own while someone has the page open.

How the chart gets into the page also splits along library lines. The four Plotly-based charts (Tasks 1, 2, 3, and 5) are embedded as live, interactive figures — panning, zooming, and hovering all still work directly in the saved HTML file, with the Plotly library either bundled inline or loaded from a CDN. The two Matplotlib-based charts (Tasks 4 and 6) are static: the finished figure is saved to an in-memory PNG and embedded as a base64-encoded image, so it displays correctly everywhere but does not support the same in-browser interaction. This split lines up with which charts need non-Latin legend text — Tasks 4 and 6 explicitly configure Matplotlib's font-family fallback to include fonts that can render Hindi, Tamil, and Japanese glyphs, something the Plotly-based tasks never need to do, since Plotly leaves text rendering to the browser itself.

The overall workflow for a visitor is otherwise the same across all six pages: open the page, and depending on the time of day, either see the chart with its filters summarised above it, or see a notice explaining when to come back. Aside from Task 2's live clock, there is no ongoing refresh — each page's gated state reflects the IST time at the moment its notebook was run, not a continuously updating clock in the browser.

11. Problems Faced

Missing values. Several columns needed for filtering — Category, Installs, Size, Rating, Android Version, Content Rating — are not fully populated in the raw files, and a missing value in any one of them would break the corresponding numeric conversion or comparison later on. Every notebook solves this by calling dropna on exactly the subset of columns that task depends on, rather than dropping any row with any missing value anywhere in the file. Task 1 goes a step further, also screening out rows where a required field is only a blank string once stripped of whitespace, since a plain missing-value check does not catch that case.

Duplicate records. Two of the six notebooks (Tasks 4 and 6) call drop_duplicates() before any other cleaning step, while the other four do not do this explicitly. Since both of those tasks build a time trend by summing installs per month, an exact duplicate row would silently inflate that month's total, which is a more consequential kind of error for a trend chart than for a simple filter-and-rank chart.

Incorrect data types. Rating, Reviews, Installs, and Price all arrive from the CSV as text, because of stray non-numeric values mixed into otherwise numeric columns. Every notebook routes these fields through pd.to_numeric with errors="coerce", which turns anything unparseable into a missing value instead of raising an error, and then drops the rows that come out empty.

Cleaning the Installs column. Installs is stored as text like "10,000+," which cannot be compared numerically until the comma and the trailing plus sign are both removed. The same three-step pattern — strip commas, strip plus signs, convert to numeric — appears with only minor variation in all six notebooks.

Size conversion. App size is recorded as a string like "19M" or "509k," or as "Varies with device" for apps that ship different sizes per device. The three tasks that filter on Size (1, 5, 6) run a small conversion function that reads the suffix, divides kilobyte values by 1024 to bring them onto the same megabyte scale, and returns a missing value for anything it cannot parse; Task 3 instead keeps only rows already recorded in megabytes and drops the rest.

Date parsing. Last Updated arrives as a text date and has to become an actual date object before its month can be checked (Task 1) or before it can be used as a monthly time axis (Tasks 4 and 6). Wherever this matters, the column is passed through pd.to_datetime, after which pandas' period-based grouping makes it straightforward to aggregate by month.

Android version conversion. Android Ver, used only in Task 3, is not a clean number — it can read as a range or as "Varies with device." The notebook handles this with a regular expression that pulls out the first number-dot-number pattern it finds and discards anything that does not match at all.

Filtering order. With five or six independent conditions active in some tasks, applying filters one at a time and printing the row count after each step (visible throughout Tasks 1, 2, 4, and 6 in particular) made it far easier to catch a filter that was unexpectedly too strict, or too loose, than combining every condition into one large boolean expression from the start and only checking the final result.

Translation handling. Three tasks need specific category names to display in another language purely for the legend or hover text, while the rest of the filtering and aggregation logic still has to run against the original English category code. Each of these tasks solves this with a small mapping dictionary consulted only when the chart is drawn, so translation never touches how the data is filtered or grouped. That approach is not applied with perfect consistency across tasks, though: Task 4's Beauty/Business/Dating dictionary is complete, translating all three into Hindi, Tamil, and German, while the same three-entry dictionary in Task 5 defines a German value for Dating but sets it to the untranslated string "Dating," so Dating displays in English there. Task 6's separate three-entry dictionary (French, Spanish, Japanese) is complete.

Rendering non-Latin legend text. The two Matplotlib-based tasks (4 and 6) explicitly set a font-family fallback chain — including fonts that support Devanagari, Tamil, and Japanese glyphs respectively — before drawing their charts. This step is not needed in the four Plotly-based tasks, since Plotly renders text through the browser rather than through Matplotlib's own font-finding, so the browser's own font stack handles non-Latin script automatically.

Dashboard time restriction and consistency. Every chart's visible window is computed with pytz.timezone("Asia/Kolkata") rather than the machine's local timezone, so the result is the same regardless of where the notebook runs. The chart HTML is only inserted into the page when the current time falls inside that task's window; outside it, the page renders a notice instead of an empty or broken chart container. Two smaller inconsistencies are worth noting: Task 2's dashboard uses a light background where the other five use a dark theme, and Task 2 alone adds a live, browser-side JavaScript clock that keeps re-checking the window every ten seconds, a level of interactivity the other five dashboards do not have.

File naming. Five of the six notebooks expect the main file as "Play Store Data.csv" (with a space); Task 5 expects "Play_Store_Data.csv" (with an underscore), and the same split applies to the reviews file. Running every notebook against one shared data folder means either keeping both filename variants on disk or renaming the files consistently across notebooks.

Plotly formatting. The four Plotly-based tasks needed explicit attention to dual y-axes, custom hover templates, and dark-theme colours (background, font, and grid colours on every axis and the legend) so the chart would match the rest of its dashboard rather than default to Plotly's standard light theme.

Performance. Because several filters run in sequence over the full dataset before any grouping happens, ordering the cheapest, most row-eliminating conditions first — such as dropping rows with missing required columns — reduces how much data the later, more expensive string-based filters (checking every app name for a particular letter, for instance) actually need to process.

12. Overall Insights

Taken as a set, the six tasks cover a reasonably complete set of standard business questions for an app marketplace: which categories are most popular (Task 1), how that popularity is spread geographically (Task 2), how Free and Paid monetisation compare within top categories (Task 3), how installs trend over time and where growth accelerates (Tasks 4 and 6), and how a specific quality trade-off — app size against rating — plays out once installs and review sentiment are accounted for (Task 5). Between them, the six charts touch installs, ratings, reviews, revenue, size, and time, which are the core dimensions most app-store analytics dashboards are built around.

A few structural patterns repeat regardless of which specific chart is being built. Category, Free-versus-Paid, and time are treated as the three main ways of slicing the data across the project, which reflects how naturally those three dimensions segment app-store data in general. Two structural choices are worth calling out explicitly to anyone using this dashboard for decisions rather than as a classroom exercise, since both are workarounds for real gaps in the source dataset rather than data the dataset actually contains: the geographic split behind Task 2's map is a modelled distribution, not verified country-level installs, and Tasks 4 and 6 both use each app's Last Updated date as a proxy for a true installs-over-time history the dataset does not otherwise provide. Both are reasonable, clearly-motivated workarounds, but a viewer who takes either chart as literal geographic or historical fact, rather than as an approximation built for a specific stated purpose, would be reading it wrong.

The pattern of filtering aggressively before charting — every task applies at least three conditions, and some apply six — consistently trades coverage for reliability: each chart ends up describing a smaller, more tightly defined slice of the Play Store catalogue rather than the whole thing, which makes the resulting comparisons more meaningful (comparing established, well-reviewed apps against each other, for instance) at the cost of the chart no longer representing "all apps in this category." That trade-off is reasonable for a focused, decision-oriented dashboard, but it is worth stating plainly rather than leaving implicit, since a viewer glancing at any one chart without reading its filter summary could otherwise assume it represents the full category rather than a filtered subset of it.

The time-gated visibility itself is best understood as a demonstration of building state-dependent interactive content, rather than as an analytics technique in its own right: it does not change what any chart shows, only when it is shown, and its main value in this project is forcing every task to be built as a genuinely self-contained module with its own fallback states — good practice for any dashboard component, regardless of what specific mechanism controls its visibility.

13. Conclusion

This project worked through the full cycle of turning a messy public dataset into a set of purpose-built, interactive visualizations: inspecting what a real-world CSV export actually contains column by column, cleaning fields that looked simple — a rating, a size, an install count — but were not usable in their raw text form, deciding what specific business question each of six charts should answer, and packaging the result as something a non-technical viewer could open and understand without touching any code.

On the technical side, the project involved sustained work in Python and pandas for data cleaning, filtering, and aggregation; Plotly and Matplotlib for building the six charts, across dual-axis, choropleth, bubble, stacked-area, grouped-bar, and time-series formats; and enough HTML, CSS, and in one case JavaScript to assemble each chart into a styled, self-contained dashboard page with conditional states for missing data and inactive time windows. The pytz library and IST-based time comparisons ran throughout, and the translation-mapping approach used in three tasks is a straightforward, genuinely reusable pattern for adding limited localisation to a chart without touching its underlying data pipeline.

Beyond the specific libraries, the project is really an exercise in data preprocessing discipline and business-oriented visualization design: recognising that almost no field in a real dataset is usable exactly as it arrives, that a chart is only as trustworthy as the filtering decisions behind it, and that two of the six charts needed a deliberate workaround (a modelled country split, a Last-Updated-based time proxy) simply because the source dataset does not contain everything a fully realistic version of that chart would need. Working through six tasks with a consistent structure — objective, filters, chart, time restriction, implementation, insight — also reinforced a habit of treating every visualization as something that has to justify itself: what question it answers, what data it deliberately excludes to answer that question cleanly, and what a viewer should and should not conclude from it.